

El examen tiene una duración de 1:30 horas.

Ejercicio de Programación (6p)

El concesionario de motos HarleyDawson desea implementar una aplicación web para gestionar la entrega de las motos a sus clientes. El concesionario necesita que la aplicación sea capaz de crear una nueva entrega, listar las entregas pendientes, cancelar una entrega y finalizar una entrega. Las funcionalidades específicas de la aplicación se definen a continuación:

- Funcionalidades:
 - Al ingresar a la aplicación, se mostrará una página donde se pueda registrar una nueva entrega, visualizar las entregas pendientes y visualizar las entregas finalizadas.
 - Para dar de alta una nueva entrega se introducirán los siguientes datos: matrícula de la moto, modelo de la moto, color de la moto, cilindrada de la moto, dirección del envío y nombre del cliente.
 - Una vez registrada una nueva entrega, esta debe aparecer en la lista de entregas pendientes.
 - Antes de registrar una nueva entrega, se verificará que no exista otra entrega con la misma matrícula. En caso de encontrar una coincidencia, se mostrará un mensaje de error.
 - Las entregas pendientes deben aparecer ordenadas por la cilindrada, ya que la empresa quiere entregar las motos de mayor cilindrada primero.
 - Junto a cada entrega pendiente aparecerán dos botones: un botón “Cancelar Entrega” y otro de “Entrega Finalizada”.
 - El botón de “Cancelar Entrega” dará la entrega como cancelada y actualizará la lista de entregas pendientes. Las entregas canceladas no deben aparecer más en ninguna de las listas.
 - El botón de “Entrega Finalizada” dará la entrega como finalizada y actualizará la lista de entregas pendientes y la lista de entregas finalizadas. Cuando una entrega se finaliza, debe desaparecer de la lista de entregas pendientes y pasar a la lista de entregas finalizadas.
- Cuestiones de implementación:
 - **Se deberá implementar una aplicación con interfaz web con arquitectura MVC tradicional (generando los HTML en el servidor) y con una arquitectura SPA (utilizando angular).**
 - La aplicación web tradicional estará en la raíz de la URL (<http://localhost/>) y la aplicación SPA estará en (<http://localhost/harleydawson/>).
 - No es necesario usar ningún tipo de estilo CSS ni librería de componentes.
 - La forma de mostrar el error cuando se intenta crear una entrega con una matricula existente será diferente en la arquitectura web tradicional y en la arquitectura SPA.
 - En la arquitectura web tradicional será una nueva página de error que muestre el mensaje “Moto pendiente de envío” y un botón o link para volver a la lista de reparaciones pendientes.
 - En la arquitectura SPA será una alerta de JavaScript que se mostrará con el código: `alert("Moto pendiente de envío");`.
 - El código deberá estar en inglés. Lo único que puede estar en castellano son los textos que aparecen en la interfaz de usuario.

Se pide:

- **A) Implementar la aplicación web con Spring MVC, SpringData y JPA. (2p)**
 - No es necesario escribir el fichero “pom.xml”, se puede asumir que tiene todas las dependencias correspondientes.
 - No es necesario escribir el fichero “application.properties”, se puede asumir que está conectada a una base de datos externa correctamente configurada.
 - No es necesario escribir la clase Application.
 - Es necesario escribir TODOS los demás ficheros de la aplicación.

- No es necesario incluir los imports en los ficheros Java.
- No es necesario implementar los getter y setter de las clases Java. Se pueden dejar indicados con un comentario.
- **B) API REST (0.5p)**
 - Se debe implementar una API REST respetando los principios de diseño: formato de las URLs, uso de los códigos de estado, cabeceras, métodos, etc.
 - No debe existir lógica duplicada con la arquitectura MVC tradicional.
- **C) Implementar el frontend usando Angular (2p)**
 - Se puede asumir que se dispone de un proyecto Angular creado con “ng new” con todos los ficheros necesarios (index.html, angular.cli, package.json, tsconfig.json, app.module.ts, etc).
 - Hay que escribir el resto de ficheros necesarios para implementar los componentes, servicios y configuración de rutas (en caso de que sean necesarias).
 - No es necesario incluir los imports en los ficheros TypeScript.
 - Se usará HTML plano en el template de los componentes (sin ninguna librería de componentes como ng-bootstrap o angular-material).
 - Se asumirá que el proxy está correctamente configurado y se pueden usar URLs relativas para acceder a la API REST del backend.
- **D) Empaquetar la aplicación en un contenedor Docker (1.5p)**
 - Escribe un fichero Dockerfile Multi-stage que construya una imagen con la aplicación (tanto backend como frontend) empaquetada.
 - Crea un script (build.sh o build.ps1 o build.bat) de construcción y publicación de la imagen docker usando ese Dockerfile. El script deberá contener el comando necesario para construir la imagen Docker con el nombre “daw/harleydawson:1.0.0” y el comando de publicación en DockerHub (se asume que el sistema está logueado en DockerHub con los permisos necesarios para publicar la imagen).
 - Se asumirá que el código Spring está alojado en una carpeta “backend” y el código Angular está alojado en una carpeta “frontend” y el Dockerfile y el script están en la carpeta raíz.
 - Se asumirá que el sistema donde se ejecute el script sólo tiene Docker instalado. No dispone de Node.js ni Maven.
 - Se asumirá que en DockerHub existen las imágenes maven:3.17.0, node:20.12.0 y openjdk-jre:21.0.0

Instrucciones de entrega

El esquema para las carpetas del examen debe ser el siguiente:

- El código del backend debe estar en una carpeta llamada “backend”.
- El código del frontend debe estar en una carpeta llamada “frontend”.
- El código de Docker debe estar en una carpeta llamada “docker”.

Solo se debe subir un fichero como entrega del examen. Dicho fichero tiene que ser un ZIP creado con “7-Zip”. El nombre del ZIP debe ser “**PPG-Examen-Junio.zip**”, donde “PPG” son las iniciales del alumno “Pepito Pérez González”.